

Time Series Event Detection in R with Harbinger



Data Analytics Lab

dal.global@grupo.cefet-rj.br | <https://eic.cefet-rj.br/dal>



The Practical Track

We start with the package's common contract, inspect data, visualize series, run detectors and evaluate results — connecting anomalies, change points, smoothing, SAX and motifs.

Contract & Setup

Package common contract, environment configuration and data loading.

Anomaly Detectors

Running simple detectors and evaluating results on real-world series.

Change Points & Transformations

Change point detection, smoothing and other time series transformations.

Inspection & Visualization

Data exploration, time series visualization and initial statistical analysis.

Hard/Soft Evaluation

Hard and soft evaluation metrics to measure the quality of detectors.

SAX, Motifs & Discords

Symbolic representation SAX, motif discovery and discord identification.

Recurring Workflow

Load data.

Choose or configure a method.

Run detection.

Inspect output structure.

Plot and evaluate against labels when available.

INPUT: **time** series dataset\$serie (**and** optionally dataset\$event)

PRE-CONDITION: harbinger package loaded; data **without** NA

STEP 1 — Instantiate the **method**: model <- method(parameters)

STEP 2 — Fit (**when** necessary): model <- fit(model, dataset\$serie)

STEP 3 — Detect: detection <- detect(model, dataset\$serie)

STEP 4 — Visualize: har_plot(model, dataset\$serie, detection, dataset\$event)

STEP 5 — Evaluate (**when** labels **are** available): evaluate(model, detection\$event, dataset\$event)

OUTPUT: data.frame **with** columns idx, event, type (**and** extra attributes **per method**)

POST-CONDITION: detection\$event **is** a logical vector aligned **to** the series index

Minimum Setup

The examples use a common seed for reproducibility.

The `harbinger` package provides detectors, evaluation, and visualization.

`daltoolbox` appears in examples that use transformations and auxiliary models.

```
source(url("https://raw.githubusercontent.com/cefet-rj-dal/harbinger/master/examples/seed.R"))  
library(harbinger)  
library(daltoolbox)
```

Data Inspection: Know Before You Detect

The choice of detector depends on the collection: size, channels, labels, and structure. Inspecting first prevents choosing an algorithm just by name.



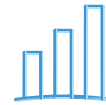
Collection Structure

Size, channels, labels, and type (univariate or multivariate).



Separate Metadata

idx, event, type, seq, and seqlen are separated from the signals by the helper.



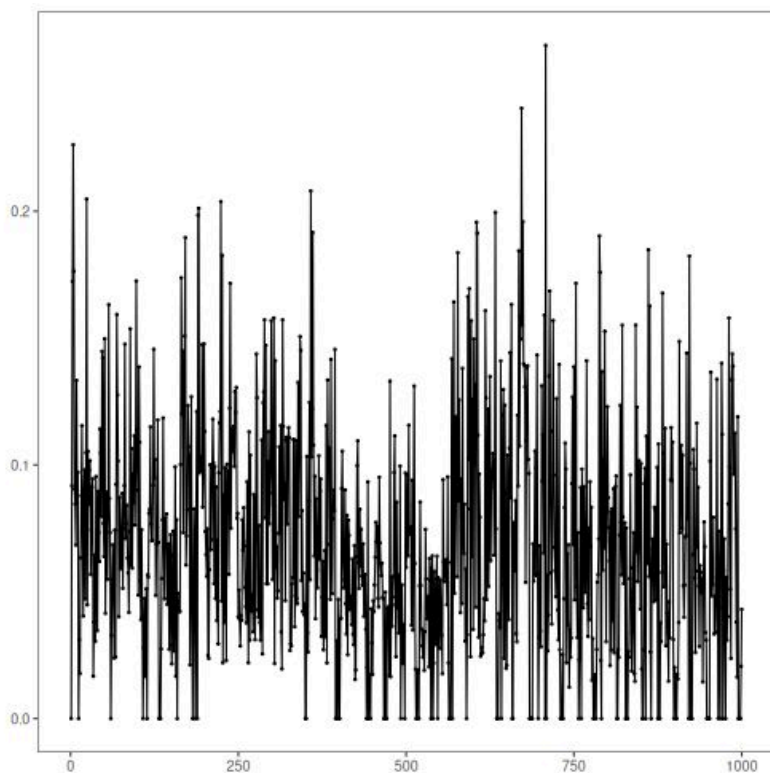
Real Benchmark

The tutorial uses A1Benchmark as an example of a real collection.

```
data(A1Benchmark)
A1Benchmark <- loadfulldata(A1Benchmark)
info <- dataset_summary(A1Benchmark)
```

Yahoo A1 Benchmark

Visual inspection of the first series in a labeled collection.



```
data(A1Benchmark)
A1Benchmark <- loadfulldata(A1Benchmark)
a1_info <- show_dataset(A1Benchmark, "A1Benchmark")
## Dataset: A1Benchmark
## Number of series: 67
## Dataset type: univariate
## Signals in the first series: value
## Column plotted with har_plot(): value
## Plot preview length: 1000 observations
plot_dataset_preview(a1_info)
```

[harbinger/examples/datasets/01-benchmarks-yahoo_benchmarks.md](https://github.com/cefet-rj-dal/harbinger/blob/master/examples/datasets/01-benchmarks-yahoo_benchmarks.md) at master · cefet-rj-dal/harbinger

Univariate and Multivariate Series

Univariate

A single signal column.

Direct input for many detectors.

Good for starting the workflow.

Multivariate

Multiple signal columns.

Requires choosing channels or combining dimensions.

Inspection guides the choice.

Labeled Data for First Contact

The first example uses a small series with an event label.

This allows understanding input, output, and visualization without yet discussing a specific detector.

```
data(examples_anomalies)
dataset <- examples_anomalies$simple
head(dataset)
```


Initial Visualization and Diagnostics

Before detecting, we visualize the series and the known events.

`har_plot()` as a diagnostic tool

More than aesthetics

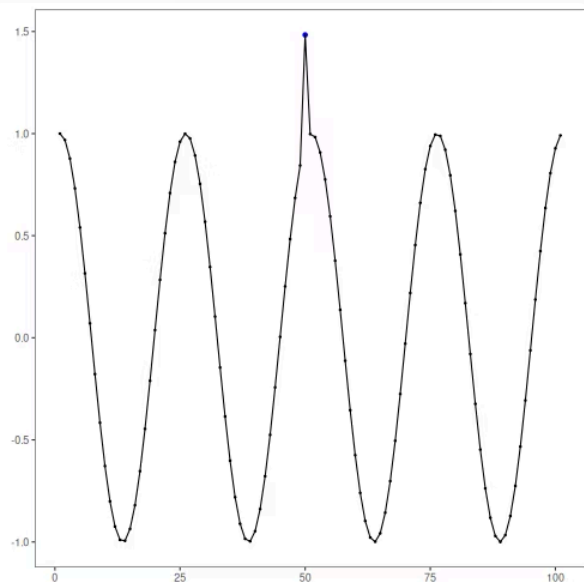
Reveals trend, bursts, level shifts, and noise before any fitting.

First check

Use before adjusting windows, thresholds, and detectors.

Overlaid events

Ground truth labels appear alongside detections for immediate visual comparison.



Common Harbinger Contract

`harbinger()` represents the common contract of the package.

A method receives a time series and returns a standardized event-oriented output.

This contract makes it easy to swap detectors without changing the entire workflow.

Events appear when we use concrete detectors, such as `hanr_arima()`, `hcp_amoc()` or `hmo_sax()`.

INPUT: time series `dataset$serie`

PRE-CONDITION: method instantiated via `harbinger()`

STEP 1 — Instantiate: `model <- harbinger()`

STEP 2 — Detect: `detection <- detect(model, dataset$serie)`

STEP 3 — Inspect: `head(detection)`

```
# idx | event | type  
#  1 | FALSE |  
#  2 | FALSE |  
#  3 | FALSE |
```

OUTPUT: standardized `data.frame` with `idx`, `event`, `type`

POST-CONDITION: identical interface for any detector in the package

Standardized Output

Minimum Structure

The output always contains `idx` (index), `event` (TRUE/FALSE) and `type` (event type). Detectors may add extra attributes.

Interchangeability

Standardization allows swapping detectors without changing evaluation and plotting code — the same interface works for any detector in the package.

```
idx | event | type
  1 | FALSE |
  2 | FALSE |
 42 | TRUE  | anomaly
```

First Anomaly Detector

`hanr_histogram()` is a simple and interpretable baseline.

The idea is to treat values in rare bins as anomalies.

It does not model temporal dynamics, but is useful as an initial reference.

Simple Baseline

Does not model temporal dynamics.
Useful as an initial reference.

Single Parameter

`density_threshold` controls the rarity threshold for bins.

Interpretable

Easy to explain: values in rare bins are anomalies.

Histogram: configure and detect

The histogram divides the series into **frequency bins** and marks as anomalies the points that fall into bins with very low density.

Main parameter: `density_threshold` — density threshold below which a point is considered rare.

Use as an **interpretable baseline** when there is no need for temporal modeling. Simple, fast, and transparent.

```
model <- hanr_histogram(density_threshold =  
0.05)  
detection <- detect(model, dataset$serie)  
head(detection)
```

INPUT: univariate series `dataset$serie`

PRE-CONDITION: **numeric** series **without** NA

STEP 1 — Build frequency histogram
of the series

STEP 2 — Calculate density of each bin

STEP 3 — Mark **as** anomaly points **in** bins
with density < density_threshold

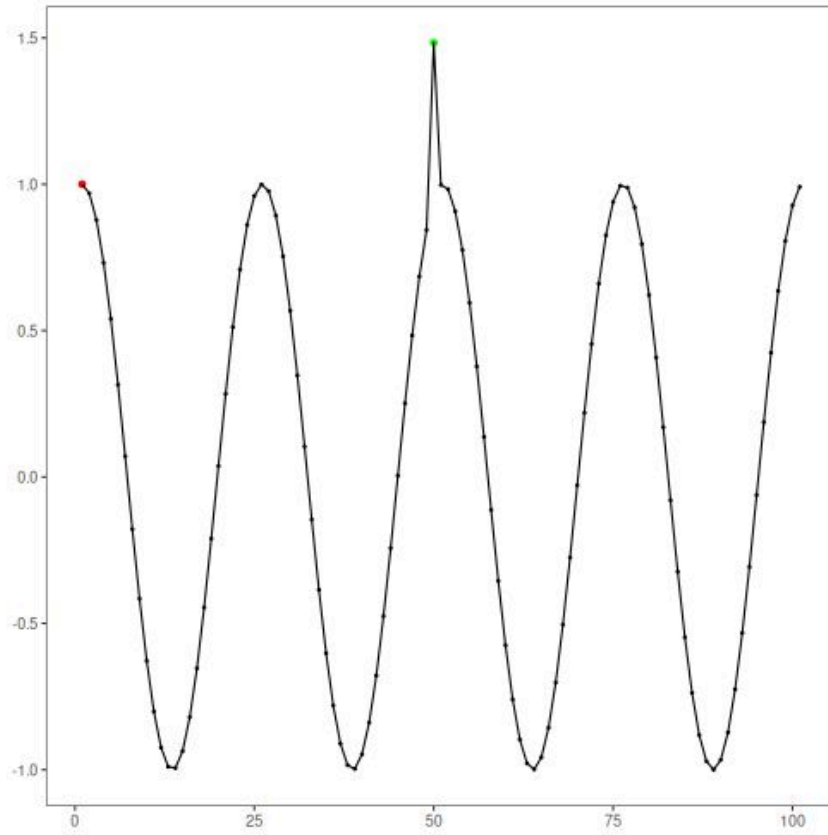
STEP 4 — **Return** indices marked **as**
event = **TRUE**

OUTPUT: detection **with** columns `idx`, `event`
(**TRUE/FALSE**), `type = "anomaly"`

POST-CONDITION: points **in** bins **with** density below `density_threshold`
are marked **as** anomaly; the **final** proportion **of** anomalies
depends **on** the distribution **and** the bins

Histogram Detector Results

Baseline detections on the labeled series.



```
library(daltoolbox)
library(harbinger)
# Load example anomaly datasets
data(examples_anomalies)
# Select a simple anomaly dataset
dataset <- examples_anomalies$simple
head(dataset)
har_plot(harbinger(), dataset$serie, event = dataset$event)
model <- hanr_histogram(density_threshold = 0.05)
model <- fit(model, dataset$serie)
detection <- detect(model, dataset$serie)
print(detection |> dplyr::filter(event == TRUE))
evaluation <- evaluate(model, detection$event, dataset$event)
print(evaluation$confMatrix)
har_plot(model, dataset$serie, detection, dataset$event)
```

Baseline Evaluation: Hard and Soft

When ground truth is available, detections can be compared against labels. The same detection can be read by different evaluators — this separates the detector's output from the judgment criterion.

```
# Direct evaluation
evaluation <- evaluate(model, detection$event, dataset$event)
evaluation$confMatrix

# Comparing evaluators
hard_result <- evaluate(har_eval(), detection$event, dataset$event)
soft_result <- evaluate(har_eval_soft(), detection$event, dataset$event)
```

What it returns

Confusion matrix (TP, FP, FN, TN) and derived metrics: precision, recall, F1.

Why compare

Hard and soft measure the same detection with different criteria. Soft F1 $\geq$ Hard F1 always.

Strict evaluation and soft evaluation

Hard evaluation

Requires direct coincidence.

Useful for well-defined point labels.

`har_eval()`

Soft evaluation

Accepts small temporal misalignment.

More suitable for events with temporal imprecision.

`har_eval_soft()`

INPUT: `detection$event` (detected logical vector), `dataset$event` (ground truth)

PRE-CONDITION: vectors of the same length; aligned by index

STEP 1 — Hard evaluation (`har_eval`):

Compare `detection$event == dataset$event` point by point

Calculate direct TP, FP, FN, TN

STEP 2 — Soft evaluation (`har_eval_soft`):

For each detected event, check if there is a label within a temporal tolerance window

Calculate TP, FP, FN with misalignment margin

STEP 3 — Calculate metrics: precision, recall, F1

OUTPUT: list with `confMatrix`, precision, recall, F1

POST-CONDITION: `har_eval_soft` assigns partial credit to detections close in time; values depend on the window `sw_size` and associations between events and detections

Model-based Anomaly Detection

`hanr_arima()` models the expected behavior of the series.

Anomalies appear as large residuals.

This example connects Harbinger to the theory of temporal models.

Temporal Modeling

ARIMA captures autocorrelation and trend of the series.

Residuals as Signal

Anomalies appear where the model errs more than expected.

Same Workflow

`fit()`, `detect()` and `evaluate()` identical to the histogram approach.

ARIMA: fit, detect and evaluate

About the method

Models the expected behavior of the time series.
Anomalies are identified as **large residuals** relative to the fitted model.

Main parameters

Order `p,d,q` selected automatically by the detector.

When to use

Stationary or trend-bearing differentiable series; no NA values.

```
model <- hanr_arima()
model <- fit(model, dataset$serie)
detection <- detect(model, dataset$serie)
              evaluate(model, detection$event,
dataset$event)
```

INPUT: **time** series `dataset$serie`

PRE-CONDITION: **numeric** series **without** NA after internal handling

STEP 1 — Fit model:

```
model <- fit(hanr_arima(), dataset$serie)
```

order (p,d,q) selected **by** `auto.arima`

STEP 2 — Detect:

refit ARIMA **with** stored **order**

compute residuals `e_t`

STEP 3 — **Convert** residuals **to** distance:

```
score_t <- har_distance(e_t)
```

STEP 4 — Apply outlier **filter**:

```
candidates <- har_outliers(score_t)
```

```
candidates <- har_outliers_check(candidates, score_t, serie)
```

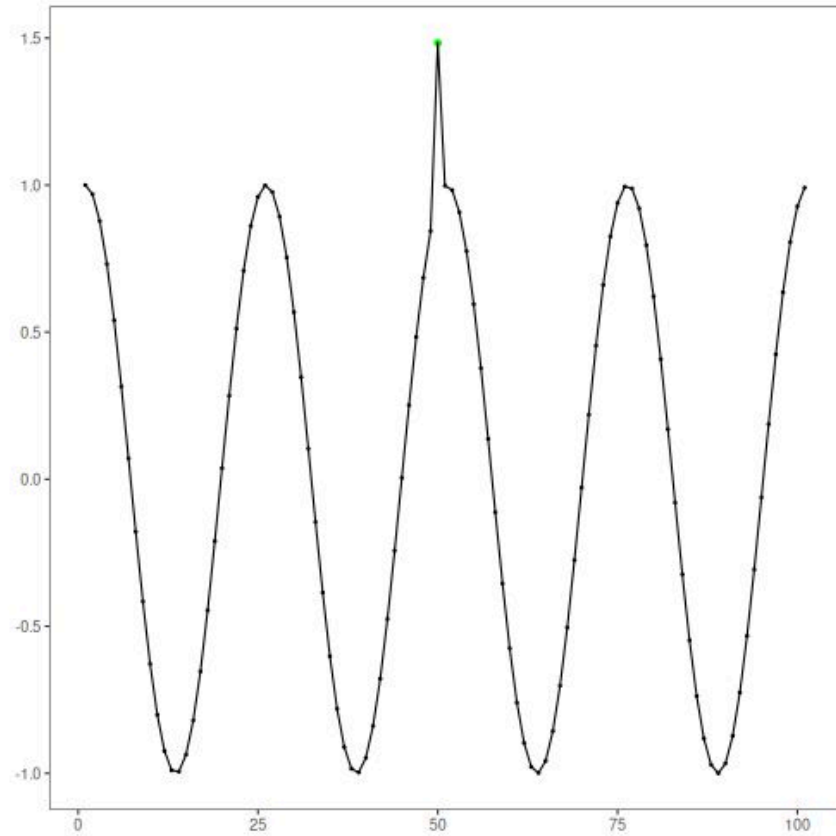
STEP 5 — Ignore **initial window** `sw_size`

and restore original indices

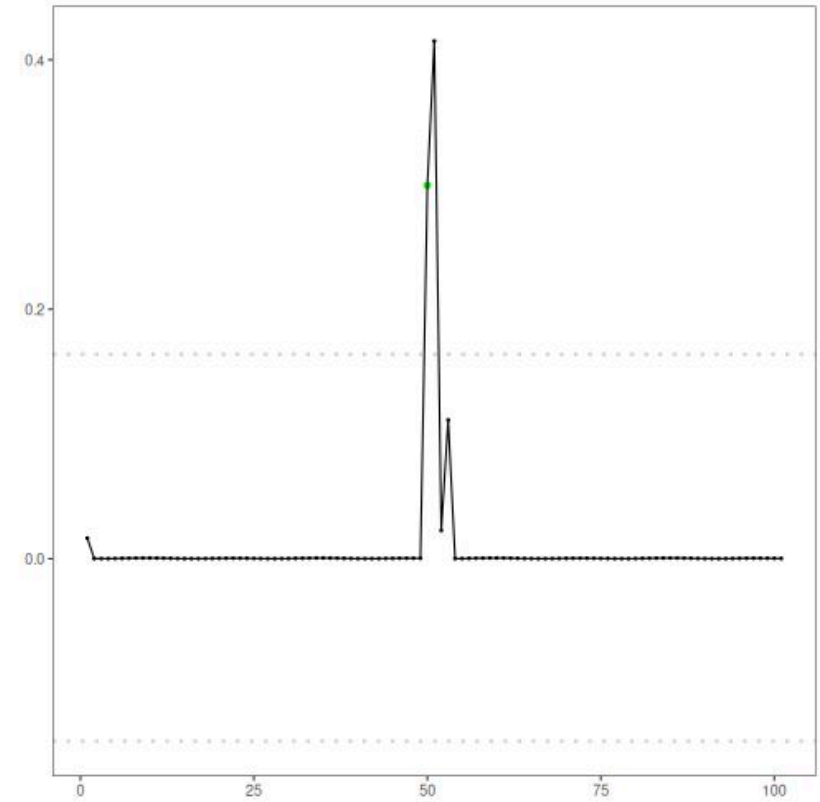
OUTPUT: detection **with** `idx`, `event`, `type = "anomaly"`

POST-CONDITION: threshold **and** candidates follow Harbinger internal utilities,
not a fixed universal `k`

Detection by ARIMA



The model detects anomalies based on deviation from the expected behavior.



The residual explains why the detector flagged certain points.

First Change Point Detector

- `hcp_amoc()` looks for a single dominant change.
- The target is not a rare isolated point, but a regime transition.
- It is a simple entry point before multiple change-point methods.

At Most One Change

Detects at most one dominant regime change.

Level Change

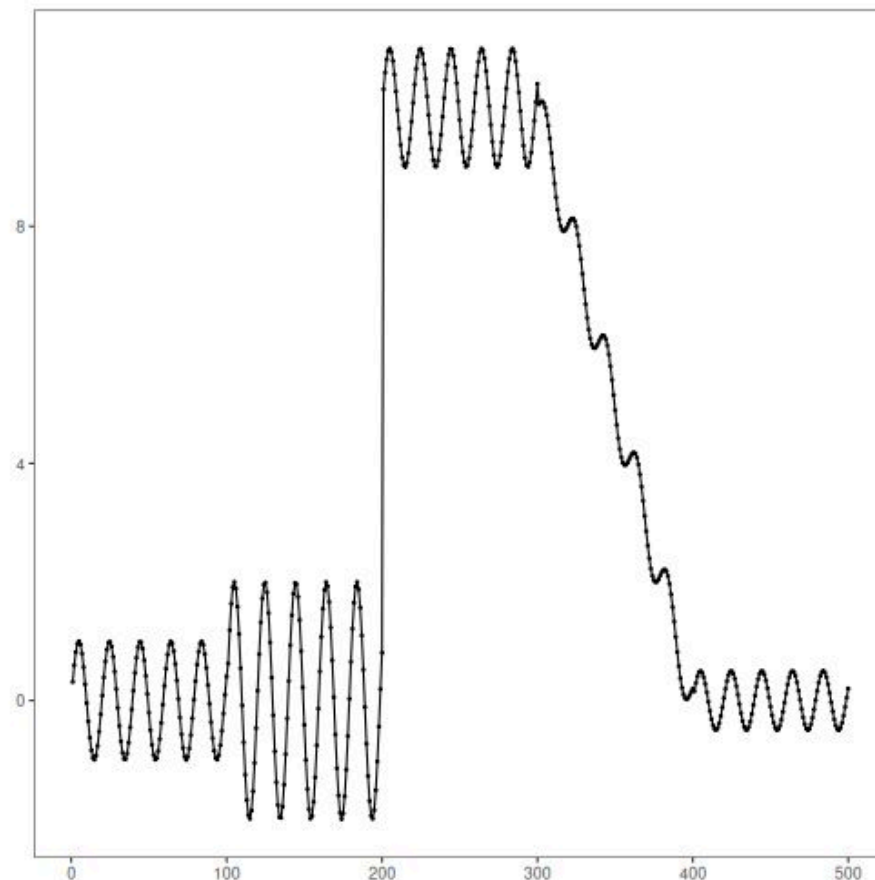
Identifies a transition between two distinct states of the series.

Starting Point

Simple entry before multiple change-point methods such as PELT or BinSeg.

Series with labeled change point

The visualization shows the series before applying AMOC.



AMOC in Harbinger

What is AMOC?

At Most One Change — detects a single dominant regime change in the time series.

When to use

Ideal for series with **one clear transition** between two distinct states, without multiple breakpoints.

Result

The algorithm uses `cpt.meanvar` with MBIC penalization and may not flag any event if the test does not support the change.

```
model <- hcp_amoc()
detection <- detect(model, dataset$serie)
har_plot(model, dataset$serie, detection,
dataset$event)
```

INPUT: **time** series `dataset$serie`

PRE-CONDITION: **numeric** series **without** NA; **at** most **one** dominant change **is** expected

STEP 1 — **Normalize** indices **and** remove NA internally

STEP 2 — Run:

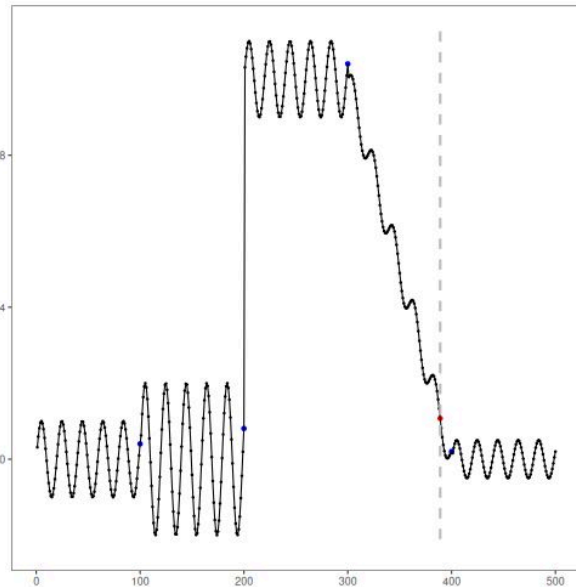
```
cpt.meanvar(serie, method="AMOC",
             penalty="MBIC", test.stat="Normal")
```

STEP 3 — Extract change point returned **by** the `cpt` object, excluding the **final** series marker **when** present

STEP 4 — Mark extracted indices **as** `event = TRUE`,
`type = "change point"`

OUTPUT: detection **with** `idx`, `event`, `type`

POST-CONDITION: detects **at** most **one** relevant change point;
may **not** flag an event if the test/penalization does **not** support the change



The chart on the side shows the changepoint detected by AMOC over the labeled series.

Transformations matter

- Before detecting, we can transform the series.
- Smoothing reveals low-frequency patterns and reduces local variation.
- The choice of transformation changes what the detector sees.

Moving average smoother

- `mas()` computes the moving average.
- Smaller windows preserve detail.
- Larger windows produce a smoother signal.

INPUT: time series `x` of size `n`; order `k = order`

PRE-CONDITION: numeric series; $1 \leq k \leq n$

STEP 1 — For `t` from 1 to `n-k+1`:

`xt[t] <- mean(x[t], x[t+1], ..., x[t+k-1])`

STEP 2 — Convert `xt` to ts object

STEP 3 — Store attributes:

`order = k`

`xi = first k-1 values of x`

`xf = last k-1 values of x`

OUTPUT: smoothed series of size `n-k+1`

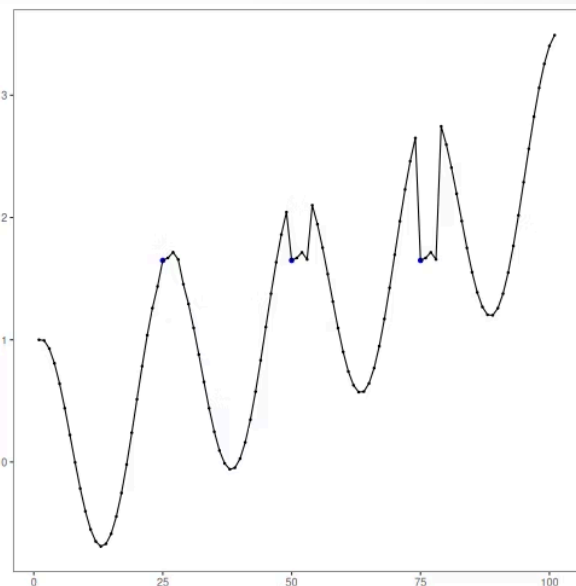
POST-CONDITION: the resulting series is shorter;

labels/events need to be realigned when using `mas()`

```
ma_5 <- mas(dataset$serie, order = 5)
```

```
ma_15 <- mas(dataset$serie, order = 15)
```


SAX and XSAX in the workflow



Symbolic Discretization

Converts continuous time series into sequences of discrete symbols, enabling efficient symbolic analysis.

- **SAX**: uses a compact alphabet of size α
- **XSAX**: allows richer encoding with an alphanumeric alphabet of up to 36 symbols (0-9A-Z)

Main Parameter

α = alphabet size (e.g.: 8 for SAX, 16 for XSAX)

When to Use

- Preprocessing for **motif discovery**
- Time series compression
- Symbolic dimensionality reduction

INPUT: time series dataset \$serie; α

PRE-CONDITION: numeric series; SAX: $2 \leq \alpha \leq 26$;

XSAX: $2 \leq \alpha \leq 36$

STEP 1 — Normalize series by z-score

STEP 2 — Compute quantiles of the normalized series
to divide values into α bins

STEP 3 — Assign each point to its bin

STEP 4 — Map bins to symbolic codes:

SAX uses A-Z

XSAX uses 0-9A-Z

OUTPUT: symbolic vector with one symbol per observation

POST-CONDITION: discretization preserves series length;

symbolic windows are created later, by detectors such as `hmo_sax()`

Motifs and Discords

The trail closes with recurring pattern analysis.

Motif discovery asks which subsequences repeat.

Discords are subsequences isolated from the rest.

Motifs

Subsequences that repeat with frequency $\geq$ minimum support. They indicate recurring patterns in the series behavior.

Marked as `type = "motif"`.

Discords

Subsequences most distant from their nearest neighbor. They indicate rare or anomalous behaviors.

Marked as `type = "discord"`.

Symbolic motif detector

What is `hmo_sax()`?

Receives the numeric series directly and applies `trans_sax()` internally before searching for recurring patterns.

- **Motifs:** subsequences that repeat with frequency $\geq$ qtd
- **Discords:** require a specific detector, such as `hdis_sax()`

Main parameters

- `a`: alphabet size (1 to 26)
- `w`: symbolic window size
- `qtd`: minimum support (≥ 2)

```
model <- hmo_sax(8, 15, 3)
model <- fit(model, dataset$serie)
detection <- detect(model, dataset$serie)
```

INPUT: **numeric** series `dataset$serie`; alphabet `a`; **window** `w`; support `qtd`

PRE-CONDITION: `a` **between** 1 and 26; `w` ≥ 1 ; `qtd` ≥ 2

STEP 1 — Apply `trans_sax(a)` internally **on** `dataset$serie`

STEP 2 — **Create** symbolic windows **with** `tspredict::ts_data(tss, w)`

STEP 3 — Concatenate symbols **from each window to** form sequences

STEP 4 — **Group** equal sequences **and** keep **groups with** `total_count` $\geq$ `qtd`

STEP 5 — Remove trivial motifs nearby **using** a band around `w/2`

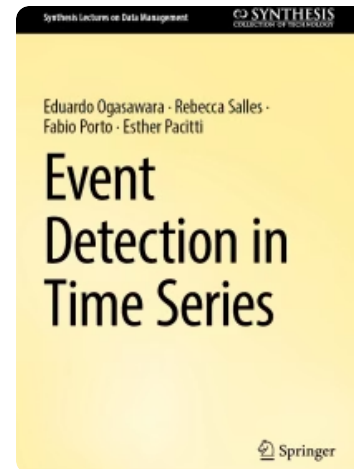
STEP 6 — Mark **first** valid occurrences **as** `event = TRUE`,

`type = "motif"`, **with** columns `seq` **and** `seqlen`

OUTPUT: detection **with** `idx`, `event`, `type = "motif"`, `seq`, `seqlen`

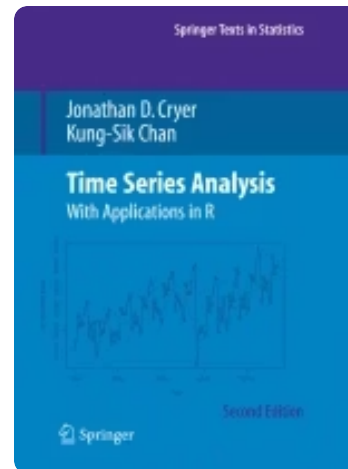
POST-CONDITION: `hmo_sax` detects recurring motifs;
discords require a **specific** detector

Bibliographic References



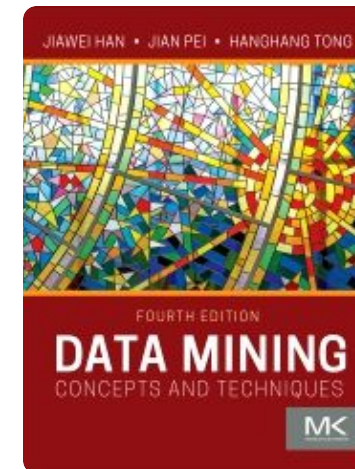
Event Detection in Time Series

Ogasawara, E.; Salles, R.; Porto, F.;
Pacitti, E. Springer Nature Switzerland,
2025.



Time Series Analysis: With Applications in R

Cryer, J. D.; Chan, K.-S. Springer, 2008.



Data Mining: Concepts and Techniques

Han, J.; Pei, J.; Tong, H. Morgan
Kaufmann, 4th ed., 2022.